

1. Explain the four major properties (pillars) of OOP in detail with examples and one diagram idea for each.

Encapsulation: Encapsulation means wrapping data and methods together and restricting access using access modifiers. Example: Bank account system where balance is private and accessed through methods. Diagram idea: Class diagram showing private data and public methods.

Abstraction: Abstraction hides internal implementation details and shows only necessary features. Example: ATM machine. Diagram idea: Interface showing only user operations.

Inheritance: Inheritance allows one class to acquire properties of another. Example: Animal → Dog. Diagram idea: Parent-child hierarchy tree.

Polymorphism: Polymorphism means one method having many forms. Example: Method overriding in subclasses. Diagram idea: Same method name behaving differently in multiple classes.

2. Compare and contrast procedural programming and object-oriented programming. Focus on focus, approach, data handling, reusability, and suitability for different types of applications.

Procedural programming focuses on functions, while OOP focuses on objects.

Approach: Procedural follows top-down approach; OOP follows bottom-up approach.

Data handling: Procedural separates data and functions; OOP binds them together.

Reusability: OOP provides high reusability using inheritance; procedural has limited reuse.

Suitability: Procedural is suitable for small programs; OOP is suitable for large and complex systems.

3. Discuss the following relationships among classes with suitable examples:

a) Association: A general relationship between two classes. Example: Teacher and Student.

b) Aggregation: A weak relationship where objects can exist independently. Example: Department and Teacher.

c) Composition: A strong relationship where objects cannot exist independently. Example: Car and Engine.

d) Inheritance: A relationship where one class inherits properties of another. Example: Animal and Dog.

4. What is message passing in OOP? Explain how message passing helps in achieving encapsulation and modularity. Support your answer with a program-like example.

Message passing is the process of communication between objects using method calls.

It supports encapsulation by allowing access only through public methods.

It supports modularity by dividing program into independent objects.

Example:

```
class A { void show(){ System.out.println("Hello"); } } class Main { public static void main(String[] args){ A obj = new A(); obj.show(); } }
```

5. Define meta-class. Explain its role in OOP languages with examples of languages that support meta-classes. How does a meta-class differ from a normal class?

A meta-class is a class of a class, meaning it defines behavior of other classes.

Role: It controls how classes are created and behave.

Languages: Python, Smalltalk support meta-classes.

Difference: Normal class creates objects, meta-class creates classes.